

15_ [++고급기능-1

2101080 정진용

1.

실습과제 1

- 일반 함수와 람다식의 차이를 자세히 설명하시오. 일반 함수로 정의하는 대신에 람다식을 사용할 때 장점은 무엇인가?

(ANSWER)

일반 함수와 람다식의 가장 근본적인 차이는 익명성과 정의 위치에 있습니다. 일반 함수는 고유한 이름을 가지며 특정 네임스페이스나 클래스 범위에 독립적으로 정의되지만, 람다식은 이름이 없는 익명 함수로서 함수를 호출하거나 변수에 대입하는 코드 위치에 직접 인라인(inline)으로 작성됩니다. 이러한 인라인 정의 방식은 STL 알고리즘에 전달할 조건자(predicate)나 간단한 콜백 함수처럼 일회성으로 사용되는 짧은 로직을 구현할 때, 코드의 다른 부분에 별도의 함수를 선언하고 정의하는 번거로움을 없애주어 코드의 간결성을 높이고 참조의 지역성을 강화합니다. 즉, 함수의 기능과 사용처가 한눈에 들어와 가독성과 유지보수성이 크게 향상됩니다. 더 나아가 람다식의 가장 강력한 장점은 캡처(capture) 기능에 있습니다. 람다식은 대괄호 []를 이용해 자신이 선언된 외부 함수의 지역 변수들을 값이나 참조로 '포착'하여 함수 내부에서 사용할 수 있으며, 이는 일반 함수가 매개 변수나 전역 변수를 통해서만 외부 데이터에 접근할 수 있는 것과 대조되는 특징으로, 복잡한 클래스나 구조체를 정의하지 않고도 특정 상태를 기억하고 동작하는 상태 저장(stateful) 함수 객체를 매우 손쉽게 만들 수 있게 해줍니다.

2.

실습과제 2

- 23페이지 예제처럼 함수에 인자로 람다식을 사용하는 예제를 인터넷에서 찾아서 실행해보고 소스코드를 자세히 설명하시오.

코드.

```
// std::function, std::cout, std::string 사용을 위한 헤더 파일 포함
#include <functional>
#include <iostream>
#include <string>

// 1. 일반(named) 함수 정의
int some_func1(const std::string& a) {
    std::cout << "Func1 호출! " << a << std::endl;
    return 0;
}

// 2. 함수 호출 연산자 operator()를 오버로딩한 '함수 객체(Functor)' 정의
struct S {
    void operator()(char c) { std::cout << "Func2 호출! " << c << std::endl; }
};

int main() {
    /*
     * std::function: 호출 가능한 모든 것을 감싸는 템플릿 클래스.
     * 일반 함수, 함수 객체, 람다식 등 다양한 형태의 호출 가능한 개체를
     * 동일한 방식으로 저장하고 사용할 수 있게 해준다.
     */

    // 'f1'에 일반 함수(some_func1)의 주소를 저장
    std::function<int(const std::string&)> f1 = some_func1;

    // 'f2'에 함수 객체(S의 인스턴스)를 저장
    std::function<void(char)> f2 = S();

    // 'f3'에 람다 표현식을 저장
    std::function<void()> f3 = []() { std::cout << "Func3 호출! " << std::endl; };

    // std::function 객체에 저장된 callable들을 동일한 문법으로 호출
    f1("hello"); // some_func1 호출
    f2('c');     // S() 객체의 operator() 호출
    f3();        // 람다식 호출
}
```

콘솔.

```
Microsoft Visual Studio 디버그
Func1 호출! hello
Func2 호출! c
Func3 호출!

C:\Users\spick\source\repos\opencv\x64\Debug\vvvvvvvvvv.exe(프로세스 25736)이(가) 0 코드(0x0)와 함께 종료되었습니다.
이 창을 닫으려면 아무 키나 누르세요...
```

3.

실습과제 3

□ 함수 포인터와 `std::function` 객체의 차이를 자세히 설명하시오.

(ANSWER)

함수 포인터와 `std::function`의 가장 큰 차이는 저장할 수 있는 대상의 범위와 안전성에 있습니다. 함수 포인터는 C-스타일의 순수한 포인터로, 시그니처가 일치하는 일반(전역) 함수의 메모리 주소만 저장할 수 있는 반면, `std::function`은 일반 함수뿐만 아니라, 함수 객체(Functor)와 캡처 기능이 있는 람다 표현식 등 호출 가능한 모든 것을 담을 수 있는 유연한 클래스 템플릿 객체입니다. 이러한 유연성 덕분에 `std::function`은 타입에 안전하며 포인터 사용으로 인한 오류를 방지해 주지만, 내부적으로 추가적인 메커니즘을 사용하므로 순수한 함수 포인터에 비해 약간의 성능 오버헤드가 발생할 수 있습니다. 결론적으로, 함수 포인터는 가볍고 빠르지만 기능이 제한적인 반면, `std::function`은 약간의 비용을 감수하고 훨씬 뛰어난 다형성과 안정성을 제공하는 현대적인 C++의 대안입니다.

4.

실습과제 4

- 23페이지 예제처럼 인터넷에서 `std::function` 클래스를 함수에 매개 변수에 사용한 예제를 찾아서 실행해보고 소스코드를 자세히 설명 하시오.

코드.

```
#include <iostream>
#include <functional> // std::function을 사용하기 위해 반드시 포함해야 합니다. [cite: 223]
#include <string>

// 계산을 수행하고 결과를 출력하는 고차 함수(Higher-order function)
// 세 번째 인자로 어떠한 연산을 수행할지 'std::function' 객체로 전달받습니다.
void calculate(int a, int b, std::function<int(int, int)> operation) {
    int result = operation(a, b); // 전달받은 연산(operation)을 실행합니다.
    std::cout << "결과: " << result << std::endl;
}

// 1. 일반 함수: 덧셈
int add(int a, int b) {
    return a + b;
}

// 2. 일반 함수: 뺄셈
int subtract(int a, int b) {
    return a - b;
}

int main() {
    int a = 10, b = 5;

    std::cout << a << " + " << b << " 연산" << std::endl;
    calculate(a, b, add); // 'add' 함수를 인자로 전달

    std::cout << a << " - " << b << " 연산" << std::endl;
    calculate(a, b, subtract); // 'subtract' 함수를 인자로 전달

    std::cout << a << " * " << b << " 연산" << std::endl;
    // 3. 람다 표현식: 곱셈
    calculate(a, b, [](int x, int y) { return x * y; }); // 곱셈 람다를 인자로 전달

    return 0;
}
```

콘솔.

```
Microsoft Visual Studio 디버그 × + -
10 + 5 연산
결과: 15
10 - 5 연산
결과: 5
10 * 5 연산
결과: 50
C:\Users\spick\source\repos\opencv\x64\Debug\vvvvvvvvvv.exe(프로세스 12404)이(가) 0 코드(0x0)와 함께 종료되었습니다.
이 창을 닫으려면 아무 키나 누르세요...
```

`calculate` 함수는 두 정수와 함께 `std::function<int(int, int)>` 타입의 `operation` 객체를 매개변수로 받는데, 이 `operation` 객체는 두 개의 `int`를 인자로 받아 `int`를 반환하는 모든 호출 가능한 것을 담을 수 있습니다. 따라서 `main` 함수에서는 미리 정의된 `add`와 같은 일반 함수 포인터를 전달할 수도 있고, 곱셈 연산처럼 별도의 함수 정의 없이 `[](int x, int y){ return x*y; }`와 같은 람다 표현식을 즉석에서 작성하여 전달할 수도 있습니다. `calculate` 함수는 내부적으로 `operation(a, b)` 코드를 통해 어떤 기능이 전달되었든 동일한 방식으로 실행하여 결과를 얻으며, 이처럼 `std::function`을 통해 수행할 연산 자체를 파라미터로 주입하는 방식은 단 하나의 함수를 매우 유

연하고 확장성 있게 만드는 좋은 방법을 보여줍니다.